@CACHEPLANE/ANGULAR · ENTERPRISE GUIDE

The Enterprise Guide to Agent Streaming in Angular

Ship LangGraph agents in Angular — without building the plumbing

cacheplane.io · 2026

Contents

01 The Last-Mile Problem

02 The agent() API

03 Thread Persistence & Memory

04 Interrupt & Approval Flows

05 Full LangGraph Feature Coverage

06 Deterministic Testing

CHAPTER 1

The Last-Mile Problem

Chapter 3: The Last-Mile Problem

Your LangGraph backend is solid. The agent graph handles tool calls correctly, memory persists across turns, and streaming responses flow cleanly through your FastAPI or Node middleware. You fire up the demo, watch tokens stream into the terminal, and everything looks exactly right.

Then you wire it to Angular.

This is where teams lose weeks.

Zone Pollution Is Structural, Not Configurational

The first symptom is subtle: the UI feels sluggish, change detection runs are exploding in DevTools, and memory climbs steadily during long agent responses. The instinct is to reach for `runOutsideAngular`. That helps — but it doesn't solve the underlying problem.

`EventSource`, the browser primitive that powers SSE, is patched by zone.js at application bootstrap. Every `message` event it dispatches enters the Angular zone automatically. With a REST response, that's one event, one change detection cycle. With a streaming agent response, you're receiving dozens to hundreds of discrete `message` events per second — each one triggering a zone task, each zone task potentially scheduling a change detection pass. You haven't configured anything wrong. The architecture of zone.js and the architecture of SSE are simply incompatible at streaming throughput.

Wrapping your `EventSource` subscription in `runOutsideAngular` moves the event handling out of the zone, but now you own the re-entry problem. Every token update that touches the DOM needs to be explicitly brought back into the zone at precisely the right moment. Do it too eagerly and you're back to thrashing. Do it too conservatively and your UI lags perceptibly behind the stream.

The Change Detection Timing Mismatch

Angular's change detection model is synchronous and pull-based. The framework walks the component tree, reads current state, and updates the DOM. This works exceptionally well for request-response patterns where state settles before the template renders.

LLM token streams are fundamentally asynchronous and push-based. A 500-token response arrives as 500 separate events across several seconds. Each event mutates your accumulator state. If you're using `ChangeDetectionStrategy.OnPush` — which you should be — none of those mutations trigger re-renders automatically unless you're managing `markForCheck` yourself on every event. If you're not on `OnPush`, you're paying the full change detection cost on every token.

Neither option is acceptable at scale without deliberate infrastructure around it.

Signals Don't Automatically Fix This

The Signals API is the right direction, but it introduces its own friction with streaming. A `WritableSignal` updated on every SSE `message` event will notify all reactive consumers on every token. For a long agent response with tool call interleaving, you're creating a high-frequency reactive pulse that propagates through the entire signal graph. Without explicit batching or debounce strategies, Signals and SSE share the same fundamental problem as zone-based change detection — they just fail differently.

Why Your RxJS Patterns Don't Transfer

Teams reach for `fromEvent(source, 'message')` and expect the familiar RxJS composability to handle everything. It handles a lot — but the operator chains you built for REST responses don't account for multi-event agent protocols. LangGraph streams `metadata`, `updates`, `messages/complete`, and error events as distinct message types within a single SSE connection. Mapping that to a clean observable that maintains agent state, accumulates tokens, handles tool call boundaries, distinguishes stream errors from agent errors, and supports reconnection is non-trivial. Every team is building this from scratch.

The Real Cost

The production gap is expensive and consistent. Teams build zone-patch wrappers, token accumulator services, custom reconnection logic, and stream-type discriminators — then rebuild them on the next project. The demo worked because it ran outside a

production Angular application with real component trees, real OnPush hierarchies, and real error conditions.

The backend is not the problem. The missing layer is a production-grade streaming adapter built specifically for how Angular actually works — not how it works in isolation, but under load, with Signals, with OnPush, and with the full complexity of a multi-turn agent protocol on the wire.

That layer is what this guide builds.

CHAPTER 2

The agent() API

Chapter 3: The agent() API

The core primitive in `@cacheplane/angular` is `agent()` — a factory function that returns a structured ref containing typed signals wired directly to a LangGraph agent stream. You call it once, bind the signals in your template, and the component reacts to every streamed token without a subscription, a zone trigger, or an accumulation buffer in sight.

What agent() Returns

`agent()` returns an `AgentRef` object with four signals:

```
interface AgentRef {
  messages:    Signal;
  isStreaming:  Signal;
  error:        Signal;
  interrupt:    Signal;
}
```

Each signal is updated incrementally as chunks arrive over the stream. `messages()` reflects the full conversation state at every emission — already assembled, already typed. `isStreaming()` flips to `true` the moment a run begins and back to `false` on completion or error. `error()` and `interrupt()` surface exceptional states that would otherwise require you to wire up parallel error channels manually.

Configuring the Transport with provideAgent()

Before `agent()` can do anything, the agent endpoint and stream transport need to be registered. That happens in `provideAgent()`, which you add to your application providers:

```
bootstrapApplication(AppComponent, {
  providers: [
    provideAgent({
      url: 'https://your-langgraph-deployment/runs/stream',
      transport: 'sse',
    }),
```

```
  ],
});
```

`provideAgent()` configures a single shared transport layer — SSE by default, with WebSocket support available via the `transport` option. It also accepts authentication headers and retry policies at this level, keeping that configuration out of individual components entirely.

Why Signal-Native Design Works with OnPush

Angular's `OnPush` change detection skips components unless an input reference changes or an event originates from within the component tree. The traditional workaround for async data — `async` pipe or manual `markForCheck()` calls — adds noise and often breaks in subtle ways under zoneless configurations.

Signals sidestep this entirely. When a signal value changes, Angular's reactive graph marks only the affected views as dirty. `agent()` produces standard Angular signals, which means `OnPush` components react to streamed tokens automatically, without zone involvement, without `markForCheck()`, and without the `async` pipe acting as a syntactic intermediary.

Template Binding Without async Pipe

Because `messages()`, `isStreaming()`, `error()`, and `interrupt()` are signals, you call them directly in the template:

```
@Component({
  selector: 'app-chat',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    @for (msg of agent.messages(); track msg.id) {

    }
    @if (agent.isStreaming()) {

    }
  `,
})
export class ChatComponent {
  readonly agent = inject(AgentService).ref;
}
```

No `async` pipe. No `subscribe()`. No lifecycle hooks managing teardown. The template reads signal values and Angular handles the rest.

The Contrast

The hand-rolled equivalent requires roughly 60–80 lines: an `HttpClient` call configured for streaming, an `EventSource` or `fetch` with `ReadableStream`, manual chunk parsing, a `BehaviorSubject` to accumulate tokens, a `takeUntilDestroyed` teardown, and explicit `markForCheck()` calls on every emission. You also write the error and interrupt handling yourself.

With `agent()`, that entire surface area collapses to three lines of meaningful code:

```
const ref = agent();
ref.send({ role: 'user', content: input });
// bind ref.messages() in template
```

The complexity didn't disappear — it moved into the library, where it belongs. What remains in your component is only the code that describes your feature.

CHAPTER 3

Thread Persistence & Memory

Chapter 6: Thread Persistence & Memory

Production agent applications are not stateless. Users close tabs, return the next morning, and expect the conversation to resume exactly where it stopped. Building that experience requires more than a reliable backend—it requires a disciplined contract between LangGraph's checkpoint system and your Angular frontend.

How threadId Flows from LangGraph to Angular

When LangGraph processes a run under a `MemorySaver` configuration, it persists the conversation graph state to a checkpoint store keyed by a `thread_id`. That identifier is the anchor for the entire session. On the Angular side, the agent library exposes this value through a reactive `threadId` signal that updates as soon as the backend confirms checkpoint creation. Every subsequent request that includes this `thread_id` in its config resumes from the last saved state—no reconstruction, no replaying events.

Treat the `threadId` signal as your source of truth. Do not generate identifiers client-side and hope for synchronization. Let the backend issue the ID, observe it, and persist it.

Persisting threadId on Creation

The `onThreadId` callback fires exactly once per new thread, immediately after the backend returns a checkpoint-backed ID. This is your window to write to `localStorage`:

```
provideAgent({
  streamUrl: '/api/agent/stream',
  onThreadId: (id) => {
    const threads = JSON.parse(localStorage.getItem('agent_threads') ??
'[]');
    threads.unshift({ id, created: Date.now(), label: 'New conversation'
});
    localStorage.setItem('agent_threads', JSON.stringify(threads));
    localStorage.setItem('agent_active_thread', id);
  },
  initialThreadId: localStorage.getItem('agent_active_thread') ??
undefined,
})
```

The `initialThreadId` input tells the agent to hydrate from an existing checkpoint on mount rather than starting a new thread. When provided, the agent skips the initialization handshake and immediately enables streaming against the existing state. If the ID is `undefined`, a fresh thread begins and `onThreadId` fires again.

Building a Thread List UI

A thread switcher is straightforward in signal-based Angular. Read the thread list from `localStorage` into a component signal, render it, and call `setThreadId()` when the user selects an entry. Clearing history means removing the entry locally and optionally calling a backend deletion endpoint—though MemorySaver's in-memory default does not expose one out of the box.

Wire up a "New conversation" button that clears `agent_active_thread` from storage and reinitializes the agent component. Since `initialThreadId` is an input, destroying and recreating the component is the cleanest reset path.

How MemorySaver Maps to the Frontend Lifecycle

`MemorySaver` holds checkpoints in process memory, making it suitable for development and single-instance deployments. Each `thread_id` maps to a serialized graph snapshot. The frontend lifecycle mirrors this directly: create a thread ID on first run, persist it, restore it on return, discard it on explicit reset. In production, swap `MemorySaver` for a persistent store—Postgres via `@langchain/langgraph-checkpoint-postgres` is the standard choice—without changing any frontend code.

Production Checklist

Before shipping thread persistence, verify the following:

- **Does your thread list handle deleted or expired server-side threads gracefully?** A stored `thread_id` that no longer exists on the server will return an error or empty state. Catch that condition, remove the stale entry from localStorage, and prompt the user to start a new conversation.
- Does switching threads cancel any in-flight stream before initiating the restored one?
- Is the thread label editable, or are users left with unlabeled UUIDs?
- Does clearing local history also revoke the server-side checkpoint where your infrastructure supports it?

Thread persistence is one of those features that feels optional until a user loses an hour of work. Build it correctly from the start.

CHAPTER 4

Interrupt & Approval Flows

Chapter 7: Interrupt & Approval Flows

Agents that interact with external systems — firing off emails, mutating database records, executing privileged queries — cannot operate as fire-and-forget processes in an enterprise context. Execution must be pauseable, inspectable, and resumable with explicit human intent. LangGraph's `interrupt()` primitive and `@cacheplane/angular`'s reactive surface for it give you exactly that, without polling loops or bespoke WebSocket plumbing.

How LangGraph interrupt() Works

When a LangGraph node calls `interrupt(payload)`, graph execution halts at that checkpoint. The payload is an arbitrary object you define — typically the action description, affected resource identifiers, and any data the reviewer needs to make a decision. The graph persists this state to its checkpointer and waits. Nothing proceeds until a resume command arrives with the correct thread ID and checkpoint reference.

The resume payload follows a typed contract: `{ action: "approve" | "edit" | "cancel", data?: unknown }`. On `approve`, the graph continues with the original node inputs. On `edit`, your modified `data` is injected, replacing what the node would have used. On `cancel`, LangGraph short-circuits the remaining path and routes to whatever terminal state you've configured for rejected flows.

The Interrupt Signal in Angular

`@cacheplane/angular` exposes this checkpoint state directly on the agent reference. The `interrupt` property is a computed signal that starts as `null` and transitions to an `InterruptState` object the moment the backend checkpoint is written:

```
const agent = injectAgentRef({ threadId });
const interrupt = agent.interrupt; // Signal


const handleApprove = () =>
  agent.resume({ action: Command.RESUME, data: { decision:
"approve" } });
```

```
const handleEdit = (corrected: unknown) =>
  agent.resume({ action: Command.RESUME, data: { decision:
"edit", payload: corrected } });


const handleCancel = () =>
  agent.resume({ action: Command.RESUME, data: { decision: "cancel" } });
```

No subscriptions. No manual HTTP calls to a resume endpoint. The signal reactivity handles template updates automatically — when `interrupt()` fires on the backend, your component re-renders with the interrupt context populated.

The Prebuilt Interrupt Panel

`` accepts the interrupt signal and your handlers, rendering a structured approval UI with the action description, resource context, and three action buttons. You bind it, you wire the handlers, and you're done. No template scaffolding, no building approval modals from scratch:

For teams with custom design systems, the panel exposes slot-based overrides without requiring you to reimplement the state machine behind it.

Edge Cases Worth Handling

Navigation mid-interrupt: The graph checkpoint persists server-side regardless of client state. On return to the thread, the interrupt signal will re-populate from the checkpoint. Guard your route exit with a confirmation dialog if the interrupt represents a time-sensitive action.

Session expiry: If the user's session expires while a checkpoint is pending, re-authentication should restore the thread context. Ensure your auth interceptor reattaches credentials before the agent ref re-initializes — otherwise the resume call will 401 against an unauthenticated thread.

Cancel with partial state: Cancel does not roll back side effects that occurred before `interrupt()` was called. If a node partially wrote records before pausing, your cancel handler needs to trigger compensating logic — either through a dedicated LangGraph cancel node or a separate cleanup endpoint. Treat `interrupt()` as a pre-execution gate, not a transaction boundary.

Interrupt flows are where the difference between a prototype and a production agent becomes visible. Get the pause-resume contract right, and the rest of the approval UX is straightforward to build on top of it.

CHAPTER 5

Full LangGraph Feature Coverage

Chapter 4: Full LangGraph Feature Coverage

Basic chat streaming is a solved problem. The real engineering challenge begins when your graph does something more interesting—calls a tool, delegates to a subagent, or needs to rewind to a prior checkpoint. Most Angular LLM libraries handle the simple case and then quietly stop working. This chapter documents how `@cacheplane/angular` exposes the full LangGraph feature surface through a coherent signal-based API.

Tool Call Streaming

When a LangGraph node invokes a tool, the graph emits a structured sequence of events: the tool call intent, intermediate streaming deltas, and the final tool result. Rather than requiring you to parse raw SSE frames and reconstruct this sequence manually, `@cacheplane/angular` surfaces tool invocation lifecycle through the agent ref directly.

The `toolCalls` signal on the agent ref updates reactively as each tool call progresses. You get the tool name, input arguments as they stream in, execution status, and the final output—all typed, all reactive. Your component never needs to touch the underlying event stream to render a "Searching the web..." indicator or display structured tool output inline.

This matters because tool call parsing is subtle. Argument deltas arrive as partial JSON strings. Parallel tool calls interleave. A naive implementation collapses these into noise. The library handles reconstruction and ordering internally.

Subgraph Event Propagation

LangGraph supports nested graphs, and real production agents use them. Events emitted inside a subgraph are namespaced by graph path, which means consuming them correctly requires understanding the event hierarchy.

`@cacheplane/angular` flattens subgraph events into the parent stream while preserving their origin metadata. If you care about which subgraph produced a message—for routing UI panels, for logging, or for debugging—you have access to the full event path. If

you don't care, the default signal behavior aggregates everything cleanly without requiring you to configure traversal logic.

Time Travel

LangGraph's checkpoint system allows you to rewind graph state to any prior node execution and re-run from that point. This is not a niche feature—it's the foundation of any agent UI that allows users to correct mistakes or explore alternative paths.

The agent ref exposes a `rewindTo(checkpointId)` method. Calling it terminates the current stream, restores the specified checkpoint, and begins re-streaming from that node. Signals update exactly as they would on an initial run. Your component code doesn't need to distinguish between a first run and a rewound run—the reactive contract is identical.

DeepAgent Multi-Agent Coordination

DeepAgent introduces orchestrator-subagent topology at the stream level. An orchestrator dispatches work to specialized agents and aggregates their outputs. From the stream consumer's perspective, this produces interleaved events from multiple graph instances.

`@cacheplane/angular` maps each active agent to its own scoped signal context. The orchestrator's agent ref exposes a `subagents` signal—a reactive map of active agent IDs to their respective state signals. You can bind a list of subagent components to this map and let Angular's `@for` loop handle the rest. Agent addition, completion, and failure all propagate as signal updates without manual subscription management.

The `onCustomEvent` Hook

LangGraph nodes can emit arbitrary structured events outside the message channel. These are the primitives for generative UI, analytics instrumentation, and intermediate state reporting.

The `onCustomEvent` hook accepts a typed handler that fires whenever a matching event arrives in the stream. You define the event schema; the library routes and deserializes the payload. The handler can write directly to a component signal, dispatch to a store, or trigger a side effect. Custom events are not mixed into the message stream—they arrive as a clean parallel channel.

Why Full Coverage Is Non-Negotiable

The failure mode to avoid is a two-tier implementation: the library handles the simple case, and your team writes custom SSE parsing for everything else. That split is where bugs live and where onboarding breaks down. An integration layer is only valuable if it holds across the full feature surface. When it doesn't, engineers route around it, and you've paid the abstraction cost without capturing the benefit.

CHAPTER 6

Deterministic Testing

Chapter 7: Deterministic Testing

Agent UIs introduce testing challenges that standard Angular component tests don't prepare you for. Streaming responses are asynchronous and stateful. LLM outputs are non-deterministic by definition. Tool calls can arrive mid-stream, get interrupted, or fail silently. If your component tests make real network calls to an LLM backend, you've already lost — slow CI pipelines, flaky outcomes, and zero ability to reproduce the specific edge case that broke production on a Friday afternoon.

The solution is full control over the data layer. Every signal value, every SSE event, every state transition should be scripted before the test runs.

MockStreamTransport

`MockStreamTransport` replaces your HTTP or WebSocket transport layer with a deterministic event queue. You script a sequence of SSE-style events — token chunks, tool call payloads, interrupt signals, error frames — and the transport emits them synchronously or on a controlled microtask schedule. No server required.

```
const transport = new MockStreamTransport([
  { type: 'token', value: 'The capital' },
  { type: 'token', value: ' of France is Paris.' },
  { type: 'done' }
]);
```

Events can be queued to emit immediately or after a configurable tick, giving you precise control over partial-stream states. You can script an interrupted stream by placing an `interrupt` event mid-sequence, or trigger an error frame at any position to verify your component's error boundary behavior.

---

createMockStreamResourceRef()

For components that consume stream state through Angular Signals, `createMockStreamResourceRef()` gives you direct write access to the signals your component reads. Instead of driving state through the transport layer, you set signal values directly and assert on the resulting DOM.

```
const ref = createMockStreamResourceRef({
  status: signal('streaming'),
  tokens: signal('Partial respon'),
  toolCalls: signal([]),
  error: signal(null)
});
```

This is the right primitive for isolated component tests. You're not testing the streaming pipeline — you're testing that your component renders correctly given a specific signal state. Keep the concerns separated.

Testing Each Agent State

Every meaningful agent state needs a dedicated test. Set the signal values, run `fixture.detectChanges()`, and assert on the DOM.

- **Streaming in progress**: `status: 'streaming'`, partial token string, verify the loading indicator is visible and the stop button is enabled.
- **Stream complete**: `status: 'complete'`, full token string, verify the stop button is gone and copy affordances are present.
- **Interrupt pending**: `status: 'interrupt'`, verify the confirmation UI renders with the correct pending action payload.
- **Error state**: `status: 'error'`, populate the error signal, verify your error component receives the correct message and a retry action is available.

Tool call rendering follows the same pattern. Populate `toolCalls` with a scripted payload and assert that the correct tool UI component is instantiated with the right inputs. Generative UI output — dynamic component injection based on tool response schema — should be tested by scripting the full tool result payload and verifying that `NgComponentOutlet` or your equivalent mechanism mounts the expected component.

Thread switching tests should use `createMockStreamResourceRef()` to swap the active thread reference and confirm the component clears its previous state correctly before rendering the new thread's history.

TestBed Setup

```
TestBed.configureTestingModule({
  imports: [AgentChatComponent],
  providers: [
    { provide: STREAM_TRANSPORT, useValue: new MockStreamTransport([]) },
    { provide: AgentStreamService, useClass: MockAgentStreamService }
  ]
});
fixture = TestBed.createComponent(AgentChatComponent);
ref = createMockStreamResourceRef({ status: signal('idle'), tokens:
signal('') });
fixture.componentRef.setInput('streamRef', ref);
fixture.detectChanges();
```

The Benchmark

Agent component tests should run completely offline and complete in under 100ms each. If a test exceeds that threshold, it's making real I/O, using real timers, or waiting on something it shouldn't know about. Treat a slow test as a design signal: your component has a seam that isn't properly abstracted. Fix the abstraction, not the timeout.